

Таймаут для методов бизнес-логики

Механизм таймаута позволяет задать методу бизнес-логики (далее по тексту "БЛ") следующие настройки:

1. Предельное время ожидания ответа от сервиса-исполнителя, где выполняется метод БЛ. Эта настройка задаётся через Genie в опции [Call timeout](#).
2. Предельное время выполнения метода БЛ в рабочем процессе сервиса-исполнителя. Эта настройка задаётся через Genie в опции [Execution timeout](#).

При выборе таймаута следует учитывать [рекомендации](#).

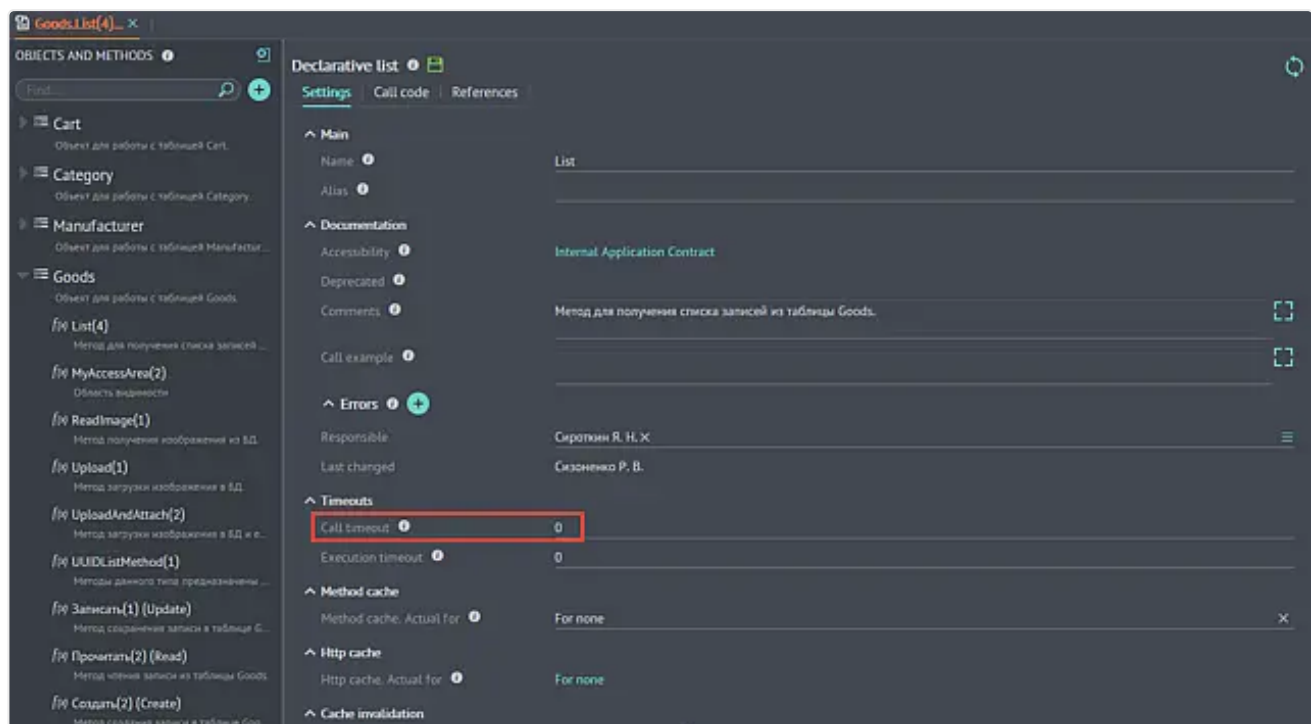
Таймаут может быть увеличен за счет операции разрешения имён с помощью DNS. Об этом подробнее читайте [здесь](#).

По умолчанию на стороне диспетчера задан [таймаут сетевых подключений](#).

Конфигурация таймаута в Genie

Call timeout

Опция задаёт предельное время ожидания ответа от сервиса-исполнителя, где выполняется метод БЛ. Отсчет времени начинается с момента, когда вызывающий сервис отправляет запрос на сервис-исполнитель. Данный таймаут работает только при [межсервисном синхронном вызове](#).



Значение опции **Call timeout** задаётся в мс. Если установлено 0, то используется значение по умолчанию — **5 минут**.

Значение для опции выбирают кратное 100 мс, что связано с точностью таймаута сетевых соединений. Также значение должно быть меньше, чем [Execution timeout](#). При выборе таймаута следует учитывать [рекомендации](#).

На сервисе-исполнителе запрос может находиться длительное время в очереди на выполнение. Если оно превышает установленный **Call timeout**, тогда:

- запрос удаляется из очереди на выполнение;
- в логи вызывающего сервиса записывается сообщение:

Sending denial of service 502 due to expiration of the query relevance

Переопределение значения опции с помощью метода SetTimeout

Значение опции **Call timeout** можно переопределить для конкретного вызова метода БЛ с помощью метода [sbis.EndPoint.SetTimeout](#) (в Python) или [sbis::blcore::EndPoint::SetTimeout](#) (в C++).

Рассмотрим пример. Методу "А" в опции **Call timeout** задано значение 500 мс. При вызове метода "А" использован метод `SetTimeout(1000)`. В результате таймаут составит 1000 мс.

Наследование значения опции

Значение опции **Call timeout** может быть унаследовано от другого метода БЛ при соблюдении условий:

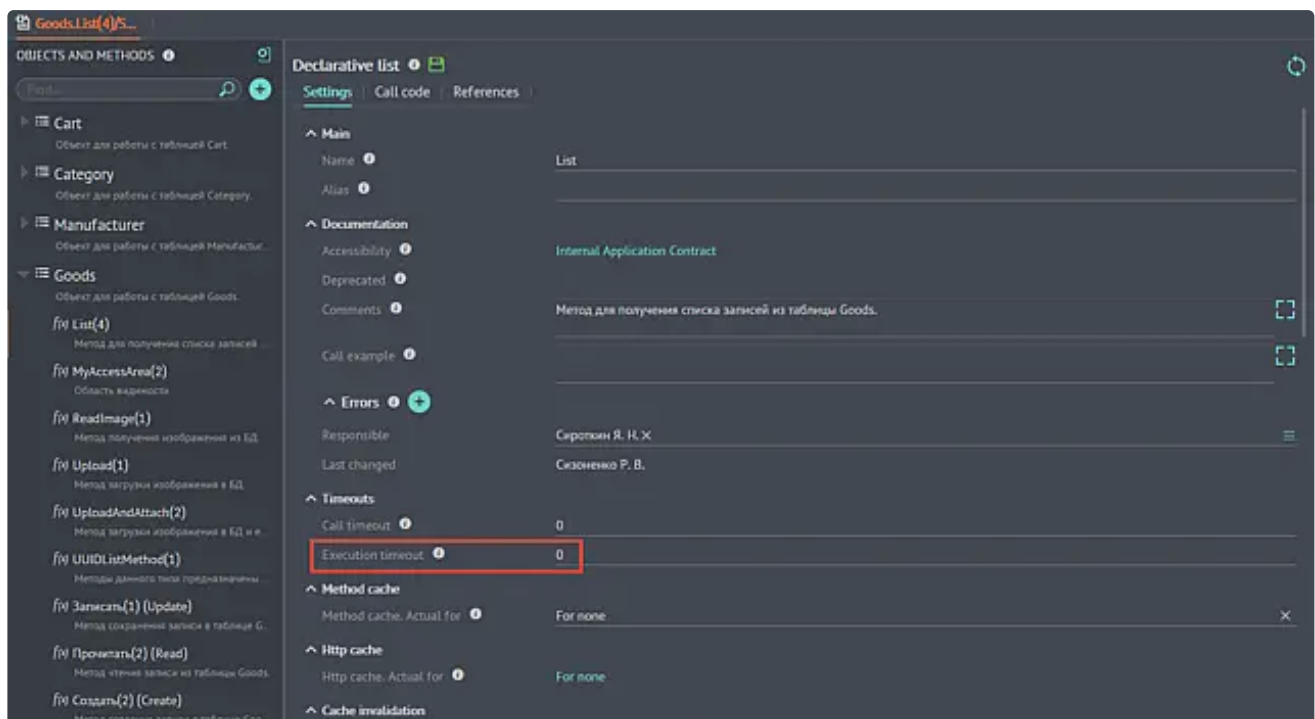
1. Методы принадлежат разным сервисам приложения.
2. Метод "А" вызывает метод "В".

При наследовании больший приоритет отдаётся таймауту от вызывающей стороны, а при его отсутствии — собственным настройкам метода БЛ. Как это работает:

- Если в методе "А" не задан **Call timeout**, и при вызове метода "А" использован [SetTimeout\(\)](#), то для метода "В" будет задан таймаут из `SetTimeout()`.
- Если в методе "А" задан только **Call timeout**, то это значение будет использовано в качестве таймаута для метода "В".
- Если в методе "А" не задан **Call timeout**, то методу "В" будет установлен таймаут согласно его собственным настройкам.

Execution timeout

Опция задаёт предельное время выполнения метода БЛ в [рабочем процессе](#) сервиса-исполнителя. Время выполнения отсчитывается с момента, когда координатор рабочих процессов передал метод БЛ на выполнение одному из рабочих процессов. При этом не учитывается время ожидания метода БЛ в очереди на выполнение.



Данный таймаут работает при синхронных/асинхронных [межсервисных \(удалённых\)](#) и [внутрисервисных](#) вызовах.

Значение опции **Execution timeout** задаётся в мс. Если установлено 0, то используется значение по умолчанию — **24 часа**. Начальное значение установлено в сервисе "[Управление облаком](#)" в параметре "Ядро.Сервер приложений.МаксимальноеВремяОбработкиЗапроса".

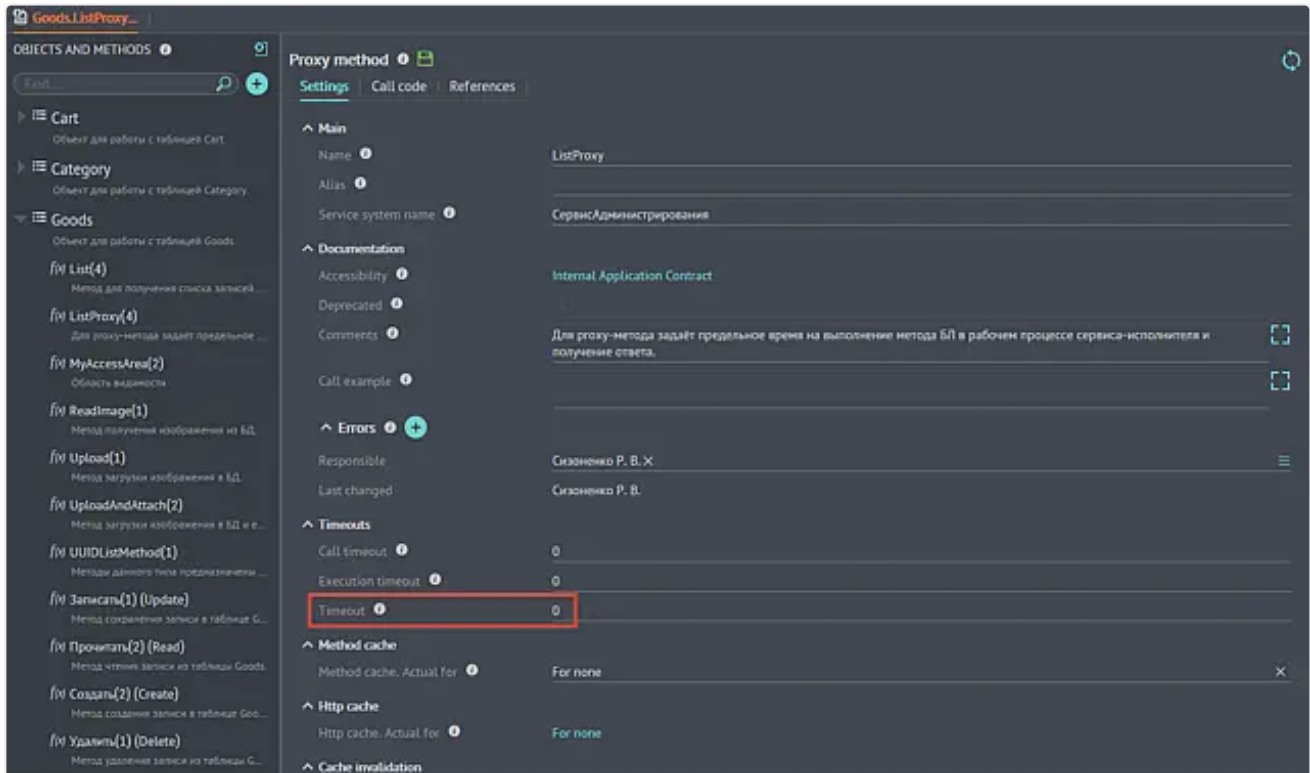
Значение для опции **Execution timeout** выбирают кратное 100 мс, что связано с точностью таймаута сетевых соединений. Также значение должно быть больше, чем [Call timeout](#).

Когда в рабочем процессе появляется метод БЛ с превышением **Execution timeout**, это считается фатальной ошибкой в работе сервиса. При этом в [logax](#) сервиса будет сохранена соответствующая запись. Сам рабочий процесс сервиса может быть завершён, о чём подробнее вы можете прочитать [здесь](#).

Execution timeout не работает для сервисов, функционирующих в режиме со встроенным пулом. В этом режиме для сервиса существует единственный рабочий процесс, который завершать запрещено. Чтобы перевести сервис в режим со встроенным пулом, в настройках сервиса установите параметр "Ядро.Сервер приложений.ТипОсновногоПула" в значение `embedded`.

Timeout (только для proxy-метода)

Для [proxy-метода](#) задаёт предельное время на выполнение метода БЛ в рабочем процессе сервиса-исполнителя и получение ответа.



Значение опции имеет наивысший приоритет над тем, что устанавливается с помощью метода [sbis.EndPoint.SetTimeout](#) (Python) или [sbis::blcore::EndPoint::SetTimeout](#) (C++).

Значение опции задаётся в мс. Если установлено 0, то используется значение по умолчанию — **5 минут**.

Рекомендации для выбора таймаута

Следует аккуратно подходить к выбору значений таймаута. Его наступление считается ошибкой и сопровождается генерацией исключения, что является нежелательным событием в системе СБИС.

При синхронном вызове метода БЛ рекомендуемый **Call timeout** составляет **5 секунд**. Значения меньше 5 секунд не рекомендуются.

Если нужен **Call timeout** с большим значением, используйте [асинхронный вызов](#) метода БЛ или, как его альтернатива, сервис [DWC](#).

Увеличение таймаута за счет операции разрешения имён с помощью DNS

Для разрешения имён DNS используется такое же значение таймаута, как и на сетевое подключение. В результате, таймаут вызова метода БЛ может произойти через удвоенный промежуток времени. Последнее в компании "Тензор" практически не встречается.

Таймаут сетевых подключений

Для сетевых подключений, в том числе межсервисных вызовов методов БЛ, на диспетчере по умолчанию задан таймаут в **20 минут**. В случае превышения лимита, происходит следующее:

- диспетчер закрывает сетевое подключение;
- вызывающей стороне возвращает HTTP-ответ со статусом 408 Request Timeout;
- на вызывающей стороне генерируется исключение класса [sbis.error.TimeoutError](#) (при работе в Python) или [sbis::rpc::TimeoutError](#) (при работе в C++).

- на сервисе-исполнителе продолжается выполнение метода БЛ; после выполнения метода, ответ не передаётся вызывающей стороне.

Глобальная настройка этого таймута задаётся в сервисе "[Управление облаком](#)" в разделе **Домены** → **Балансировщики HTTP** → **Конфигурация** с помощью параметра `route_proxy_read_timeout`.

